Week 4: Personalized Rating Prediction

Reading Project: Near-Optimal Real-Time Personalization with Simple Transformers

Goal for the week

Implement multinomial logistic regression for one user's movie ratings, then test whether a single attention layer can learn genre interactions that linear scores cannot. Use only movie genres as inputs. Fit both models separately for each user, with no shared weights or pretraining across users.

1 Reading

Review Week 3's softmax, cross-entropy, and attention code.

2 The models we are building

For a fixed user a, each example is $(g_m, r_{a,m})$: the movie's 19 binary genre flags and the user's rating. Earlier ratings provide training labels, not extra features or history tokens. Both models predict one of five ratings:

$$\mathcal{R} = \{1,2,3,4,5\}, \quad k = r - 1 \in \{0,\dots,4\}, \quad p = \mathrm{softmax}(z), \quad \mathcal{L} = -\log p_k.$$

Logistic regression: learn weights for genres

Set $x = g_m \in \{0,1\}^{19}$, including the *unknown* genre flag. For user a,

$$z_k = b_{a,k} + w_{a,k}^\top g_m, \quad k = 0,\dots,4, \qquad W_a \in \mathbb{R}^{5\times 19}, \quad b_a \in \mathbb{R}^5.$$

Each rating class has its own genre weights; all five classes use the same g_m. Learn these weights from this user's training ratings only. Add no hand-built genre-interaction features.

Transformer: let genre tokens interact

For each of the $G = 19$ genres, form $t_j = e_j + g_{m,j} v_j \in \mathbb{R}^d$, with $d = 16$. Learn e_j, v_j independently for this user: the token identifies the genre and whether it is present. Keep all 19 tokens, including absent genres, in a fixed order; stack them as rows of $T \in \mathbb{R}^{G\times d}$. Use one attention head:

$$A = \mathrm{softmax}_{\mathrm{row}}\left(\frac{(TW_Q)(TW_K)^\top}{\sqrt{d}}\right), \qquad U = T + A(TW_V), \qquad z = R\,\mathrm{flatten}(U) + b.$$

Flatten rows in genre order. Learn $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$, $R \in \mathbb{R}^{5\times(Gd)}$, and $b \in \mathbb{R}^5$. All transformer parameters are fitted using this user's training data only; the user subscript is omitted above.

3 Questions

Q1. State the prediction problem

Name the user and show one movie's genre vector, rating, and class index. Explain what is available at prediction time. Give this user's chronological split cutoffs and train/validation/test counts.

User 405 (MovieLens 100K), selected by a rule fixed before fitting: the user with the most ratings among users with at least 10 distinct timestamps (737 ratings, 154 distinct timestamps). Only counts and timestamps were used.

Example (training set): *Raiders of the Lost Ark (1981)*, rating $r = 5$, class $k = r - 1 = 4$, genre vector $g_m = (0, 1, 1, 0, \ldots, 0) \in \{0, 1\}^{19}$ (Action, Adventure present; order: unknown, Action, Adventure, Animation, ..., Western).

At prediction time the model sees only the movie's 19 genre flags. This user's earlier ratings are used only as training labels; the target rating and any rating history are never inputs.

Chronological 70/15/15 split, ties kept together (cutoffs are Unix timestamps, all on 1998-01-23 UTC): train $\leq$ 885548474 (514 ratings), validation 885548475–885549192 (114), test $>$ 885549192 (109, first at 885549308). No timestamp is shared across splits.

Q2. Construct the data for both models

For Q1's example, show logistic regression's input and two transformer tokens: one present genre and one absent genre. Trace both paths to five logits, with dimensions. Derive $\partial\mathcal{L}/\partial z_k$ and explain how it updates the logistic-regression weights. Identify what the transformer learns.

Logistic regression input. $x = g_m \in \{0, 1\}^{19}$ with ones at Action (index 1) and Adventure (index 2). Then $z = Wx + b$, $W \in \mathbb{R}^{5\times 19}$, $b \in \mathbb{R}^5$: $(19) \to (5)$, 100 parameters.

Transformer tokens ($d = 16$, $t_j = e_j + g_{m,j} v_j$). Present genre (Action, $j = 1$): $t_1 = e_1 + v_1 \in \mathbb{R}^{16}$. Absent genre (Drama, $j = 8$): $t_8 = e_8 \in \mathbb{R}^{16}$. All 19 tokens are kept in fixed order and stacked into $T \in \mathbb{R}^{19\times 16}$.

Path to logits. With $A = \mathrm{softmax}_{\mathrm{row}}(QK^\top/\sqrt{16})$, $U = T + AV$ and $z = R\,\mathrm{flatten}(U) + b$:

$$T\,(19, 16) \to (Q, K, V) = (TW_Q, TW_K, TW_V)\,(19, 16) \to A\,(19, 19)$$
$$\to U\,(19, 16) \to \mathrm{flatten}\,(304) \to z\,(5).$$

Batched: $(B, 19) \to (B, 19, 16) \to (B, 5)$.

Gradient. With $p = \mathrm{softmax}(z)$ and $\mathcal{L} = -\log p_y$: $\mathcal{L} = -z_y + \log \sum_j e^{z_j}$, so

$$\frac{\partial \mathcal{L}}{\partial z_k} = -\mathbf{1}[k = y] + \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k - \mathbf{1}[k = y].$$

For logistic regression $z_k = b_k + w_k^\top g$, so $\partial\mathcal{L}/\partial w_{k,j} = (p_k - \mathbf{1}[k = y])\, g_j$ and $\partial\mathcal{L}/\partial b_k = p_k - \mathbf{1}[k = y]$. A gradient step is $w_k \leftarrow w_k - \eta(p_k - \mathbf{1}[k = y])g$: the true class's weights for present genres rise, the other classes' fall in proportion to p_k, and weights for absent genres do not move.

What the transformer learns: the genre identity embeddings e_j, the presence offsets v_j, the projections W_Q, W_K, W_V and the readout R, b (2,901 parameters). Unlike logistic regression, whose score is a sum of independent genre weights, attention mixes the tokens, so the logits can depend on genre combinations.

4 Code

Reuse Week 3's training loop; change the data, features, and output dimensions.

Task 1: build rating examples

- Reuse MovieLens 100K. Join `u.data` to `u.item`'s 19 genre flags by movie ID. IDs only group/join records. Ratings are 1–5; index 0 means one star, not zero.
- Start with one user having enough ratings for nonempty chronological 70%/15%/15% splits; keep timestamp ties together. State a selection rule based on rating counts and timestamps before fitting, not model performance.
- Use the same training movies, genre vectors, and labels for both models. All training ratings are usable; there is no 20-event history limit. The target rating never enters a model's input.

Task 2: train both models

Initialize and train each model from scratch using only the selected user's ratings. Train transformer tokens, attention, and readout end to end. Use raw-logit cross-entropy; select checkpoints by validation loss and test once. Log seed, optimizer, learning rate, batch size, epochs, parameter counts, and runtime. If time permits, repeat independently for two more users; report each.

Task 3: verify the interface and evaluate

Check input shape $(B, 19)$, token shape $(B, 19, 16)$, output shape $(B, 5)$, labels $k = r - 1$, and tiny-batch learning. Verify gradients reach the token, query/key/value, and readout parameters. Use $1 + \arg\max_k p_k$ for accuracy and $\widehat{r} = \sum_{k=0}^{4}(k+1)p_k$ for RMSE; report unregularized test loss.

Results (user 405, seed 0). Test set: 109 ratings. Config for both models: Adam, learning rate 3×10^{-3}, batch size 64, at most 300 epochs, stop after 25 epochs without a validation-loss improvement, $d = 16$. Checkpoints chosen by validation loss; test evaluated once.

Model	Params	Best ep.	Val loss	Test loss	Acc (%)	RMSE	Sec
Logistic regression	100	25 / 50	1.2327	0.8548	90.83	1.2771	0.4
Transformer	2,901	3 / 28	1.0456	0.6707	90.83	0.9223	0.4
Majority class (1 star)	–	–	–	–	90.83	–	–
Train-mean rating (2.03)	–	–	–	–	–	1.0272	–

The transformer has lower validation loss, test loss and RMSE than logistic regression, and is the only model whose RMSE beats the constant train-mean baseline. Accuracy is identical for both models and equals the majority-class rate (90.83%), so accuracy does not separate them. The test set is 91% one-star ratings against 60% in training, so test numbers are dominated by this shift. This is one user and one seed; with 109 test ratings I do not treat the gap as established.

5 Write-up

Save models, mappings, dataset version, cutoffs, and configuration. Fill the report; send the PDF **24 hours before our meeting**.

W1. One specific question (or "none after checking").

For each training movie, the 19 genre flags g become 19 tokens $t_j = e_j + g_j v_j$. One attention head computes $A = \mathrm{softmax}(QK^\top/\sqrt{d})$ and $U = T + A(TW_V)$; the flattened U goes through the linear readout to 5 raw logits. Cross-entropy $-\log p_y$ is taken on the logits, backpropagation sends gradients through R, the attention, and into e_j, v_j, and Adam updates all 2,901 parameters from this user's training ratings only. After each epoch the validation loss is computed, and the checkpoint with the lowest validation loss is kept (early stopping on validation loss). The test set is evaluated once, with that checkpoint.

W2. Explain the training process for the transformer-based method.

User 405 rated all 737 movies within about 90 minutes on one day, and the share of one-star ratings jumps from 60% in training to 91% in test. Both models tie the majority class on accuracy. How should I tell whether the transformer's lower test loss and RMSE come from learned genre interactions rather than from fitting this one session's drift toward one star? My plan is to repeat with several seeds and on the next two users by the same rule (655 and 13), whose ratings span more distinct timestamps.

W3. Repository/commit, run command, and total hours.

Repository: `https://github.com/dwc2425/transformer-personalization`, commit `c8bf3f8`. Run from `src/`: `python verify_ratings.py ../ml-100k` then `python train_ratings.py --root ../ml-100k` (user 405 chosen automatically by the stated rule; seed 0). Total hours: 4 (3 coding, 1 reviewing and explaining how it works).